

Concurrency Keywords in C++ and Python

C++ Concurrency Keywords and Constructs

Core Keywords (C++11 onward):

- thread: `std::thread` for thread creation
- mutex: `std::mutex` for mutual exclusion
- `unique_lock`, `lock_guard`: RAII-based locking
- `condition_variable`: thread synchronization
- future, promise: asynchronous result passing
- atomic: lock-free atomic operations
- `this_thread::yield`, `this_thread::sleep_for`: thread control

C++20 Coroutines:

- `co_await`: suspend execution
- `co_yield`: yield values
- `co_return`: return from coroutine

Other Utilities:

- `std::jthread`: joinable thread with automatic joining
- `stop_token` / `stop_source`: thread cancellation control
- barrier, latch: synchronization primitives
- `scoped_lock`: multiple mutex locking

Python Concurrency Keywords and Constructs

Threading (threading module):

- Thread: create new threads
- Lock, RLock: thread-safe locks
- Event, Condition, Semaphore, Barrier: synchronization tools

Multiprocessing (multiprocessing module):

- Process: separate processes for parallel execution

- Queue, Pipe: inter-process communication
- Manager: shared objects
- Lock, Semaphore: process-safe synchronization

Asyncio (Asynchronous I/O):

- async def: define asynchronous functions
- await: suspend/resume coroutine
- async for, async with: asynchronous iteration/context
- asyncio.create_task(): schedule coroutine
- asyncio.gather(): run tasks concurrently
- asyncio.Lock, Event, Condition: async-safe primitives

Summary Table

Feature	C++	Python
Thread creation	std::thread, std::jthread	threading.Thread
Locks	std::mutex, lock_guard	Lock, RLock
Condition variables	condition_variable	Condition, Event
Async support	co_await, co_yield, co_return	async, await, async def
Futures & Promises	std::future, std::promise	asyncio.Future
Shared memory	std::atomic, manual sharing	multiprocessing.Manager()
Communication	shared memory, libraries	Queue, Pipe, asyncio.Queue
Parallel execution	manual or libraries	multiprocessing, concurrent.futures